Prometheus + Grafana Monitoring on Amazon EKS

A simple guide: the concepts, the tools, how it helps in production, and a hands-on lab with custom metrics

Inside this guide	Section
Monitoring in one minute	1
Core concepts	2
Tech stack: what each tool does	3
How it fits together	4
Alerts: from rule to a human	5
How this helps in real production	6
What we build in the lab	7
Demo script: step by step, with what you will see	8
Break it, then fix it	9
Production vs this lab	10
Interview questions with short answers	11
Official references	12

Before you start. The lab runs on real AWS and costs about **$0.35 per hour** (EKS + 2 x t3.large + NAT + EBS). It takes 2 to 3 hours. Always run the clean-up step at the end.

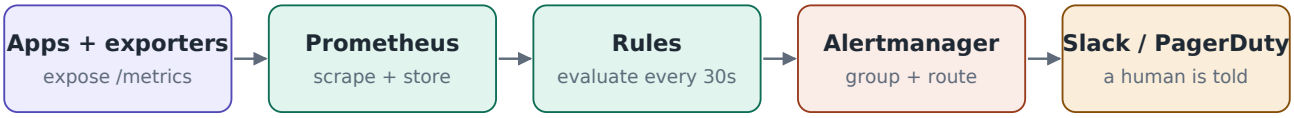
Author: Aniruddha Choudhary, Cloud / DevOps / MLOps engineer. Tested November 2026: EKS 1.35, terraform-aws-modules/eks v21, kube-prometheus-stack 92.x. Code: companion GitHub repository eks-prometheus-grafana-monitoring-lab.

1. Monitoring in one minute

Your apps run as **pods** on **nodes**. Things go wrong: an API gets slow, a pod keeps crashing, a disk fills up. Monitoring tells you **before your users do**, and shows you **where** to look.

- **Prometheus** visits every component every 15 to 30 seconds, reads its numbers, and stores them over time.
- **Grafana** turns those numbers into dashboards.
- **Alertmanager** tells a human (Slack, PagerDuty, email) when a number crosses a limit.

> **Think of a car dashboard.** The sensors are **exporters**. The car computer that records the readings is **Prometheus**. The gauges you look at are **Grafana**. The warning light and the beep are **Alertmanager**.



Grafana reads from Prometheus with PromQL to draw dashboards.

2. Core concepts

2.1 The three pillars of observability

Pillar	What it is	Answers	Example tools
Metrics	Numbers over time	Is something wrong?	Prometheus, CloudWatch
Logs	Detailed event records	What exactly happened?	Loki, ELK, CloudWatch Logs
Traces	One request's path across services	Where is it slow or failing?	Jaeger, Tempo, X-Ray

This lab covers **metrics**, which are usually the first signal in an incident.

2.2 Pull model

Prometheus **pulls**: it sends `HTTP GET /metrics` to every target on a schedule. This is called a **scrape**. Apps do not push anything. If a target cannot be reached, the scrape fails and Prometheus records `up = 0`, so a dead target is detected automatically. For short batch jobs that end before a scrape, use the **Pushgateway**.

2.3 What a metric looks like

```
orders_total{status="failed", pod="orders-app-7d9f"}  42
|__ name __| |_____________ labels _____________|   |_ value
```

Each unique **name + set of labels** is a **time series**: Prometheus stores a new value for it on every scrape.

2.4 Metric types

Type	Behaviour	Example	Typical query
Counter	Only goes up (resets on restart)	orders_total	rate(orders_total[1m])
Gauge	Goes up and down	orders_in_queue, memory used	use the value directly

Type	Behaviour	Example	Typical query
Histogram	Counts values in buckets	order_processing_seconds	histogram_quantile(0.95, ...)
Summary	Quantiles worked out in the app	rare on Kubernetes	cannot be added across pods

Label cardinality. Never use values without a limit, such as `user_id` or `order_id`, as labels. Every new value creates a new time series, and too many series can crash Prometheus.

2.5 What to monitor

Method	Use it for	Signals
Golden signals (Google SRE)	Any service	Latency, Traffic, Errors, Saturation
RED	Request-driven services (APIs)	Rate, Errors, Duration
USE	Resources (nodes, disks)	Utilisation, Saturation, Errors

2.6 Where metrics come from

Source	Meaning	Examples
Exporter	A translator for software that does not speak Prometheus	node-exporter (Linux), kube-state-metrics (Kubernetes objects), mysqld-exporter
Built-in	Software that already exposes /metrics	kubelet / cAdvisor, API server, CoreDNS, Prometheus itself
Client library	Your own app exposes custom metrics	prometheus_client (Python), Micrometer (Java), prom-client (Node.js)

2.7 The Prometheus Operator

On Kubernetes you do not edit `prometheus.yml` by hand. The **Prometheus Operator** adds new Kubernetes object types (CRDs). You describe monitoring in YAML, and the Operator rewrites Prometheus's configuration for you.

CRD	Purpose
ServiceMonitor	Scrape the pods behind this Service
PodMonitor	Scrape these pods directly, without a Service
PrometheusRule	Alert rules and recording rules
AlertmanagerConfig	Where alerts are sent

3. Tech stack: what each tool does

Tool	Role in this lab	Why it is used
Amazon EKS 1.35	Managed Kubernetes; AWS runs the control plane	Standard managed Kubernetes on AWS
Terraform + terraform-aws-modules	Builds the VPC, cluster, node group, add-ons and IAM	Repeatable, reviewable infrastructure
Managed node group	2 x t3.large worker nodes for all pods	Simple and managed by AWS
EBS CSI driver + gp3	Creates EBS disks when a PVC asks for one	Monitoring data must survive restarts
EKS Pod Identity	IAM role for the EBS CSI driver	No static AWS keys inside pods
Helm 3	Installs the monitoring stack from a values file	One command, settings kept in Git
kube-prometheus-stack	Prometheus, Operator, Alertmanager, Grafana, exporters, default dashboards and alerts	The most common Kubernetes monitoring bundle
Prometheus	Scrapes, stores (7 days) and evaluates rules	The standard for metrics (CNCF project)
node-exporter	CPU, memory, disk and network of each node	Infrastructure health
kube-state-metrics	State of Kubernetes objects: restarts, replicas, phases	Workload health
Alertmanager	Receives, groups and routes alerts	Keeps detection separate from notification
Grafana + sidecar	Dashboards; loads dashboards from ConfigMaps	Dashboards as code
Python + prometheus_client	Sample orders-app with custom metrics	Shows how developers instrument code

4. How it fits together

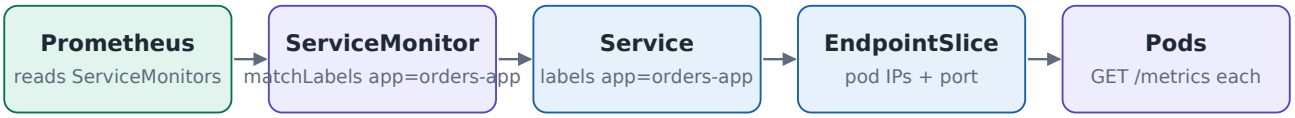
AWS: created by Terraform

- **VPC** — 3 AZs, private + public subnets, 1 NAT
- **EKS cluster 1.35** — add-ons: vpc-cni, coredns, kube-proxy
- **Managed node group** — 2 x t3.large worker nodes
- **EBS CSI driver** — Pod Identity IAM role, no keys
- **EBS gp3 volumes** — created on demand by PVCs

Kubernetes: Helm + kubectl

- **StorageClass gp3** — default, WaitForFirstConsumer
- **kube-prometheus-stack** — Prometheus, Alertmanager, Grafana
- **orders-app (demo ns)** — custom metrics on :8000/metrics
- **ServiceMonitor + Rule** — scrape config + alerts
- **Dashboard ConfigMap** — loaded by the Grafana sidecar

How Prometheus finds your app



Prometheus uses the Service only to find pod IPs, then scrapes every pod directly.

#	Match	Set by
1	Which ServiceMonitors Prometheus reads	Helm value serviceMonitorSelectorNilUsesHelmValues. Default: only those labelled release: <helm-release-name>
2	Which Service the ServiceMonitor picks	spec.selector.matchLabels must equal the Service's metadata.labels
3	Which pods sit behind the Service	Service spec.selector must equal the pod labels

What up **means.** up = 1: Prometheus scraped the target successfully. up = 0: the scrape failed. It does **not** mean "the pod is running". For that, use kube_pod_status_phase from kube-state-metrics.

5. Alerts: from rule to a human



'for: 2m' stops short spikes from waking someone up.

- **Prometheus** decides *whether* something is wrong. It evaluates each PrometheusRule every 30 seconds.
- **for** is a waiting time. The condition must stay true that long before the alert fires.
- **Alertmanager** decides *who hears about it, and how*: routing by labels such as `severity=critical`, grouping similar alerts, silences during maintenance, and inhibition (hide minor alerts when a major one fires).
- **Watchdog** is an alert that always fires on purpose. If it ever stops, the alert pipeline itself is broken.

6. How this helps in real production

Use case	How monitoring helps
Detect incidents early	Alerts on error rate and latency fire before customers complain (lower MTTD)
Faster troubleshooting	Dashboards show what changed, when and where: pod, node, release (lower MTTR)
Safe releases	Watch the error rate after each deploy and roll back fast; drives canary analysis (Argo Rollouts, Flagger)
SLOs and error budgets	Measure availability and latency, e.g. "99.9% of orders succeed"
Capacity planning and FinOps	Compare requested vs actual CPU and memory to right-size pods and nodes
Autoscaling	Custom metrics such as queue length can drive HPA (prometheus-adapter) or KEDA
Business visibility	Orders per minute and failed payments next to the technical metrics
On-call	Critical alerts to PagerDuty, warnings to Slack, silences during maintenance
MLOps / LLMOps	Model latency, prediction errors, GPU usage (DCGM exporter), token throughput

7. What we build in the lab

Component	Details
EKS cluster	monitoring-demo, Kubernetes 1.35, 2 x t3.large managed nodes
Storage	Default gp3 StorageClass using the EBS CSI driver with Pod Identity
Monitoring stack	kube-prometheus-stack; disks for Prometheus (20Gi), Alertmanager (5Gi), Grafana (5Gi)
Sample app	orders-app, 2 pods. Python code lives in a ConfigMap, so there is no image to build
Custom metrics	orders_total (Counter), order_processing_seconds (Histogram), orders_in_queue (Gauge)
Scraping	ServiceMonitor for the orders-app Service
Alerts	HighOrderFailureRate, OrdersAppDown, OrdersAppMissing
Dashboard	"Orders App" Grafana dashboard stored as a ConfigMap

Who creates what

Created by	What
Terraform	VPC, EKS cluster, node group, add-ons, IAM role for the EBS CSI driver
kubectl	StorageClass, sample app, ServiceMonitor, PrometheusRule, dashboard ConfigMap
Helm	The whole monitoring stack, configured by helm/kube-prometheus-values.yaml
Kubernetes, automatically	EBS volumes (from PVCs), Prometheus scrape config (from the Operator)

8. Demo script: step by step

Files: `terraform/`, `helm/` and `k8s/`. Tools: Terraform 1.9 or newer, AWS CLI v2, kubectl, Helm 3.

Step 1: build the cluster (Terraform, about 15 to 20 min)

```
cd terraform
terraform init
terraform plan -out tfplan
terraform apply tfplan

aws eks update-kubeconfig --name monitoring-demo --region us-east-1
kubectl get nodes -L node.kubernetes.io/instance-type
kubectl get pods -n kube-system
```

You will see: 2 Ready t3.large nodes; ebs-csi-controller, aws-node, coredns and eks-pod-identity-agent Running.

Step 2: default gp3 StorageClass

```
cd ..
kubectl apply -f k8s/storageclass.yaml
kubectl get storageclass
```

You will see: gp3 (default), provisioner ebs.csi.aws.com, WaitForFirstConsumer.

Step 3: install the monitoring stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace -f helm/kube-prometheus-values.yaml
kubectl get pods -n monitoring
kubectl get pvc -n monitoring
```

You will see: all pods Running and 3 PVCs Bound (20Gi, 5Gi, 5Gi).

Key value	Why
serviceMonitorSelectorNilUsesHelmValues: false (also podMonitor..., rule...)	Read all ServiceMonitors and rules, not only those labelled release: monitoring
serviceMonitorNamespaceSelector: {}, ruleNamespaceSelector: {}	From every namespace
retention: 7d	Keep metrics for 7 days
storage / persistence on gp3	Data survives pod restarts
kubeScheduler, kubeControllerManager, kubeEtcd: enabled: false	EKS hides the control plane; avoids false Down alerts

Step 4: open Prometheus, Grafana and Alertmanager

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-alertmanager 9093:9093
```

You will see: Prometheus at localhost:9090 (Status, Target health: all built-in targets UP); Grafana at localhost:3000 (admin / admin123) with about 25 ready-made dashboards; Alertmanager at localhost:9093 with the Watchdog alert. Run one PromQL query per box: up, then sum(rate(node_cpu_seconds_total{mode!="idle"}[5m])) by (instance).

Step 5: sample app with custom metrics

```
kubectl apply -f k8s/sample-app.yaml
kubectl get pods -n demo
kubectl port-forward -n demo svc/orders-app 8000:8000
curl -s localhost:8000/metrics | grep -E "^orders|^order_processing"
```

You will see: orders_total{status="success"}, orders_total{status="failed"}, orders_in_queue and histogram buckets. The FAIL_RATE env var (default 0.05) controls the failure percentage.

Step 6: ServiceMonitor

```
kubectl apply -f k8s/servicemonitor.yaml
kubectl get endpointslices -n demo -l kubernetes.io/service-name=orders-app
```

You will see: in Target health, search orders: serviceMonitor/demo/orders-app/0, 2/2 UP, one target per pod IP. Try: sum(rate(orders_total[1m])) by (status) and histogram_quantile(0.95, sum(rate(order_processing_seconds_bucket[5m])) by (le)).

Step 7: custom alerts

```
kubectl apply -f k8s/prometheusrule.yaml
kubectl set env deployment/orders-app -n demo FAIL_RATE=0.3     # simulate a bad release
# wait about 4 minutes, then revert:
kubectl set env deployment/orders-app -n demo FAIL_RATE=0.05
```

Alert	Condition
HighOrderFailureRate	Failed orders above 10% for 2 minutes
OrdersAppDown	up == 0 for 1 minute (scrape failing)
OrdersAppMissing	absent(up{job="orders-app"}) for 2 minutes (target silently disappeared)

You will see: Alerts page: HighOrderFailureRate goes Pending, then Firing with a value near 0.3. It then shows in Alertmanager with the label severity=critical.

Step 8: Grafana dashboard as code

```
kubectl apply -f k8s/grafana-dashboard.yaml
```

You will see: Grafana, Dashboards, Orders App: orders per second by status, failure rate (green, red above 10%), p95 processing time, and queue size per pod. The Grafana sidecar loads any ConfigMap labelled `grafana_dashboard: "1"` in the monitoring namespace. Each panel's expr is a normal PromQL query.

9. Break it, then fix it

Break 1: label mismatch

Change the ServiceMonitor to `matchLabels: app: orders-api` and apply it.

- The orders target disappears and up{job="orders-app"} returns **empty**. There is no error anywhere.
- OrdersAppDown does not fire, because there is no series at all (not up = 0). **OrdersAppMissing does fire.**
- **Fix:** make matchLabels match the Service's labels again.

Break 2: the Helm selector setting

```
helm upgrade monitoring prometheus-community/kube-prometheus-stack -n monitoring \
  -f helm/kube-prometheus-values.yaml \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=true
kubectl get servicemonitor -n monitoring --show-labels
```

- The orders target disappears, but the built-in targets stay UP: they all carry release=monitoring.
- **Fix A:** add `release: monitoring` to the ServiceMonitor labels (already done in the repo).
- **Fix B:** run helm upgrade again without the --set.

> **Lesson:** a target missing from Prometheus is almost always a **label or selector** problem, not an app problem. Check the three matches in section 4, in order.

10. Production vs this lab

This lab	Real production
helm install by hand	Argo CD / GitOps, values kept in Git
Password in the values file	External Secrets / Secret Manager, plus SSO login
kubectl port-forward	Ingress with TLS and a domain name
7 days of data, one replica	Thanos or Mimir for long-term data; 2+ Prometheus and 3 Alertmanager replicas
Alerts go nowhere	AlertmanagerConfig routes to PagerDuty and Slack
One cluster	Central Grafana for many clusters, or Amazon Managed Prometheus / Grafana

Clean-up (order matters)

```
helm uninstall monitoring -n monitoring
kubectl delete pvc --all -n monitoring     # Helm does not delete StatefulSet PVCs
kubectl delete -f k8s/sample-app.yaml
kubectl get pv                             # must be empty
cd terraform && terraform destroy
```

> Delete the PVCs **before** terraform destroy. Otherwise the EBS volumes are left behind and keep costing money.

11. Interview questions: short answers

What does kube-prometheus-stack install?

The Prometheus Operator and its CRDs, Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics, and default dashboards and alert rules for Kubernetes.

Pull vs push: why does Prometheus pull?

Prometheus controls the scrape rate, and a dead target shows up at once as up = 0. Push is used only for short batch jobs, through the Pushgateway.

Counter vs Gauge vs Histogram?

A counter only goes up (use rate()); a gauge goes up and down (queue size, memory); a histogram puts values in buckets so you can calculate p95/p99 latency with histogram_quantile().

How does a ServiceMonitor find the app?

It selects a Service by label in the chosen namespace. Prometheus reads that Service's EndpointSlice for the pod IPs and scrapes every pod directly.

Your new ServiceMonitor target does not appear. What do you check?

1) Does Prometheus read this ServiceMonitor (release label vs serviceMonitorSelector, namespace selector)? 2) Does matchLabels match the Service's labels? 3) Does the endpoint port name match a named port on the Service? 4) Does the Service have endpoints (pods Ready)?

What does up = 0 mean?

The scrape failed: wrong port or path, the app is not serving /metrics, or a network issue. It does not prove the pod is down.

Why use 'for' in an alert rule?

To ignore short spikes. The condition must stay true for that long, so people are not woken up by noise.

Prometheus vs Alertmanager?

Prometheus evaluates the rules and decides whether something is wrong. Alertmanager groups, silences and routes the alerts to people.

Why do KubeSchedulerDown and KubeControllerManagerDown fire on EKS?

AWS manages and hides the control plane, so Prometheus cannot scrape them. Turn those components off in the Helm values.

How do you keep Grafana dashboards safe?

Store them as JSON in ConfigMaps (or provisioning files) in Git, so they are recreated on every deploy instead of living only in the UI.

12. Official references

#	Reference	Link
1	Prometheus overview	prometheus.io/docs/introduction/overview/
2	Prometheus metric types	prometheus.io/docs/concepts/metric_types/
3	kube-prometheus-stack Helm chart	github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack
4	Prometheus Operator documentation	prometheus-operator.dev/docs/
5	Grafana documentation	grafana.com/docs/grafana/latest/
6	Google SRE book: Monitoring Distributed Systems	sre.google/sre-book/monitoring-distributed-systems/